Lab: Implement a Working Express Server and a User Model with Mongoose

Please note that this lab is **not graded.** In this lab exercise, you will learn how to set up a working Express server and create a user model using Mongoose. By the end of this exercise, you will have a functional backend application capable of handling user data efficiently. We encourage you to try this lab exercise independently.

🚨 **Note Before You Begin**

This is a **challenging lab** designed to help you gain real-world experience in building and running full stack applications.

You will need to:

- **Debug** your own issues,
- **Troubleshoot** errors,
- And **solve problems independently**, just like professional developers do.

⚠️ Even **small typos** in your code (like a missing comma or incorrect filename) can break your server or stop the web app from loading. Always check your code carefully!

A. Installation Requirements for Backend:

- Install Node.js: Download and Install Node.js from https://nodejs.org/en for your system. You can install Node.js by directly downloading the source code or a pre-built installer specific to your workspace platform. The version of Node.js you choose to download will come bundled with npm as the package manager.
- Test by typing "node --version" in your command prompt to view the version number returned after installation

```
$ node --version
```

- Install Visual Studio: Download and Install text editor from https://code.visualstudio.com/. A feature-rich source code editor

B. Installing the prerequisites for the full stack:

To integrate the full stack technologies and run your applications, we will need the following:

Express: To use Express in your code, you will need the express module.

MongoDB: To use MongoDB directly with Node applications, you need to add the driver, which is available as a Node module named mongoose. You would have MongoDB Community Server and for Mac or Windows installed previously.

Creating a Project using "npm init"

- Open your Visual Studio terminal or command prompt.
- Navigate to the directory "C:\mongoproj" to create your new Node.js project.
- Run **"npm init"** in terminal window. Press Enter.

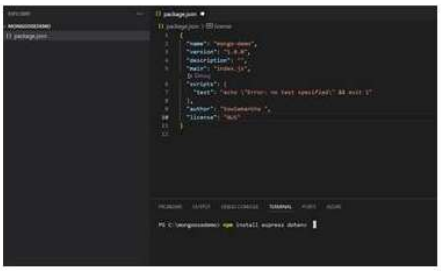
```
C:\mongoproj> npm init
```

- You will be prompted to provide information about your project, such as package name, version, etc. You can accept the default values suggested by npm. Press Enter.

- It will generate a package.json file in your current directory based on the information you provided.

Install Express.js

- Run **"npm install express dotenv"** in the terminal window. When you run npm install express, (Node Package Manager) installs the Express framework and its dependencies into your project. This allows you to start using Express in your Node.js applications.



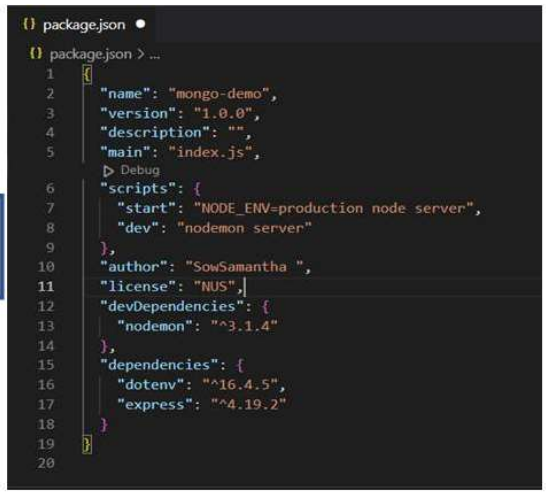
Using project dependencies – package.json

- Run **"npm i -D nodemon"** in your terminal to install nodemon as a development dependency in your Node.js project, allowing you to automatically restart your application when files change during development.

```
C:\mongoosedemo> npm i -D nodemon
```

- Open "package.json" to amend "scripts" to

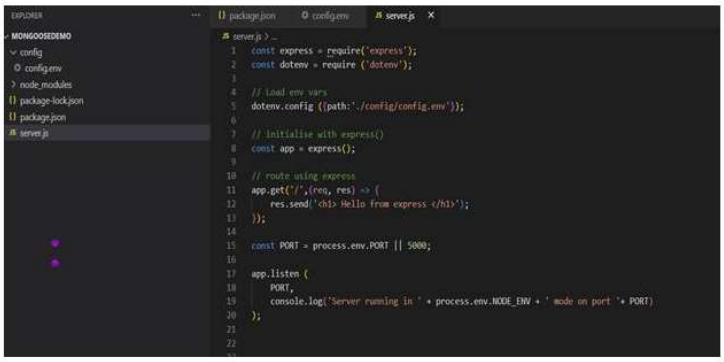
```
"scripts": {
  "start": "NODE_ENV = production node server",
  "dev": "nodemon server"
},
```



```
{} package.json ●
{} package.json > ...
{
  "name": "mongo-demo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷ Debug
  "scripts": {
    "start": "NODE_ENV=production node server",
    "dev": "nodemon server"
  },
  "author": "SowSamantha ",
  "license": "NUS",
  "devDependencies": {
    "nodemon": "^3.1.4"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
  }
}
```

Create the server.js file—the entry point of backend application

- Starting the Server: Finally, the server.js file usually includes the code to start the server listening on a specified port. This is the entry point where your backend application begins execution.
- **Create a new "server.js"** file and include the code below



```
EXPLORER
MONGOOSEDEMO
config
config.env
node_modules
package-lock.json
package.json
server.js

{} package.json   ⚙ config.env   server.js ×
server.js > ...
const express = require('express');
const dotenv = require ('dotenv');

// Load env vars
dotenv.config ({path:'./config/config.env'});

// initialise with express()
const app = express();

// route using express
app.get('/',(req, res) => {
    res.send('<h1> Hello from express </h1>');
});

const PORT = process.env.PORT || 5000;

app.listen (
    PORT,
    console.log('Server running in ' + process.env.NODE_ENV + ' mode on port '+ PORT)
);
```

- **Create a folder config and include a new file "config.env"** and include the environment code below.

Express server listening on port 11000

- Run in development mode. Run **"npm run dev"** in your terminal

```
PS C:\mongoosedemo> npm run dev

> mongo-demo@1.0.0 dev
> nodemon server

[nodemon] 3.1.4
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running in development mode on port 5000
```

- Open a web browser. In the address bar, enter http://localhost:11000 and press Enter

- What this means:
  - localhost refers to your own computer
  - 11000 is the **port number** which tells your computer the specific service to talk to

This URL address connects your browser to a local web server that's running on your machine.

⚠ **Troubleshooting & Warning**

- If the page does not load or you see an error "site can't be reached", your server is not running or not configured correctly.
- Do not proceed further until this issue is resolved.

Make sure you have

- Started the server with npm run dev.
- Have no typos in your server.js or config files.
- Have the correct port (11000) specified in your code and config.env.

- All being well, you should be presented with the default Express project landing page with "Hello from express", as shown in the following screenshot:

Hello from express

🛠 **Debugging Tip for Students**

If you encounter issues:

- **Go back and repeat all steps in Sections A and B** carefully.
- Make sure:
  - Node.js and npm are properly installed.
  - All packages like Express and Node were installed without errors.
  - The server is running with npm run dev.
- **Check for typos** in your code, especially in server.js, package.json, and config.env.
- If problems continue, **uninstall Node.js and Visual Studio Code**, then **reinstall them** and try again from the beginning.

🚫 **Do not proceed to later steps** until **http://localhost:11000** loads successfully in your browser.

Making sure this part works is critical before continuing with the rest of your project.

C. Installing the Mongoose to connect to MongoDB

- Use the connection string which you have for your Mongo Compass. You can use your Atlas Mongo database as well.
- First install the mongoose module. Run **"npm install mongoose --save"** in your terminal to install mongoose. You can read more at the website https://mongoosejs.com/
- Include a "db.js" file in config folder. Import the mongoose module using "const mongoose = require('mongoose');"
- Use mongoose.connect('mongodb://localhost:27017/') to connect to your database
- Export your module as connectDB.
- Then, update the server.js file to import the mongoose module, configure it to call the connectDB ( ) to handle the connection to the MongoDB database for the project.
- Run in development mode. Run "npm run dev" in your terminal. You should see Mongodb Connected. Congratulations!
- Proceed to define the User Schema in db.js

```
/**
 * Schema definition
 */

// Mongoose.Schema
const UserSchema = new mongoose.Schema({
    name: {
      type: String,
      trim: true,
      required: 'Name is required'
    },
    email: {
      type: String,
      trim: true,
      unique: 'Email already exists',
      match: [/.+\@.+\..+/, 'Please fill a valid email address'],
      required: 'Email is required'
    },
    hashed_password: {
      type: String
    },
    salt: String,
    updated: Date,
    created: {
      type: Date,
      default: Date.now
    }
  })

// compile User model from UserSchema
const User = mongoose.model("User", UserSchema);
```

- Create the 4 methods which contains the commands such as Find(), UpdateOnet() and DeleteMany()

```
/**
 * Methods
 */

async function createUsers (name, email, password ) {

    const user = new User({ name: name, email:email, hashed_password:password })
    await user.save();
    console.log("New user :"+ name + "Email: " + email);

}

async function listUsers() {

      let users = await User.find({}, {name:1, email:1, _id:0});
      console.log(users);

}

async function updateUsers (name, newname) {

    const user = await User.updateOne({ name: name}, {$set: {name: newname}});
    console.log("Update user :"+ name + "to: " + newname);


}


async function deleteUsers () {

    const user = await User.deleteMany({});
    console.log("Delete all users ");


}
```

- Include these methods into the connection

```
// Connection to the MongoDB database for the project.
const connectDB = async() => {

    const conn = await mongoose.connect('mongodb://localhost:27017/')
    .then( () =>{ console.log('Mongodb Connected!');   } );

    await createUsers('Jill Anne', 'JillAnne@gmail.com', 'password@1234');
    await createUsers('Bruno', 'B123@gmail.com', 'password@1234');
    await listUsers();
    await updateUsers('Bruno', "Brian Brown");
    await listUsers();

};
```

- Run in development mode. Run “npm run dev” in your terminal. You should see the documents inserted and updated in Mongodb Congratulations!
- Proceed further to modify these methods and add more functionalities!